

AWS API Gateway Lab Part 3: Create HTTP API and Route to Lambda

To Begin with the Lab

Summary of the Lab

In this part of the lab, an **HTTP API** was created in **API Gateway** to expose the backend Lambda functions as real API routes. The lab connected the frontend request flow to the three Lambda functions prepared earlier: create user, get user, and delete user. A new API named **UserAPI** was created, and routes were mapped so that POST /user invoked **createUserFunction**, GET /user/{userId} invoked **getUserFunction**, and DELETE /user/{userId} invoked **deleteUserFunction**. This layer acted as the bridge between the client application and the Lambda backend, since Lambda cannot be called directly from the public internet. The PDF also confirms that this lab uses **HTTP API** and that the next lab will compare it with **REST API**.

Understanding API Gateway in This Lab

API Gateway exists so a frontend application can send HTTP requests to a secure entry point instead of calling Lambda directly. In this lab, it acts as the middle layer that receives POST, GET, and DELETE requests and forwards each request to the correct Lambda function. This structure keeps the application secure, organized, and easy to expand later. Since the API is built as an **HTTP API**, the setup is lighter and faster for Lambda integration than a full **REST API** setup.

Example:

A mobile app wants to create a user named Kabir, read user 101, and delete user 101. Instead of talking directly to Lambda, the app sends POST /user, GET /user/101, and DELETE /user/101 to **API Gateway**. API Gateway then forwards each request to the matching Lambda function, which performs the DynamoDB operation.

Lab Steps

Step 1 – Open API Gateway

- Sign in to the AWS Management Console.
- Open **API Gateway**.
- Click **Create API**.

The screenshot shows the AWS API Gateway console. On the left, the 'APIs' link is highlighted in the navigation menu. The main area displays a table of existing APIs. In the top right corner, the 'Create API' button is highlighted with a red box.

	Name	Description	ID	Protocol	API endpoint type	Created	Security policy	API status
☐	api-intelcore1	for testing	api-intelcore1	REST	Regional	2026-05-16	TLS_1_0	Available
☐	APIControl1	for demo	api-control1	REST	Regional	2026-01-07	TLS_1_0	Available
☐	APIControl2	for demo	api-control2	REST	Regional	2026-01-07	TLS_1_0	Available
☐	apigateway2-API	Created by AWS Lambda	api-gateway2	REST	Regional	2025-02-19	TLS_1_0	Available
☐	apigateway3		api-gateway3	REST	Regional	2025-02-19	TLS_1_0	Available
☐	apigateway4		api-gateway4	REST	Regional	2025-02-19	TLS_1_0	Available
☐	awsapigateway1-API	Created by AWS Lambda	api-gateway1	REST	Regional	2025-02-19	TLS_1_0	Available
☐	awslambda1-API	Created by AWS Lambda	api-lambda1	REST	Regional	2026-05-16	TLS_1_0	Available
☐	demoapi1	demoapi	api-demo1	REST	Regional	2025-04-28	TLS_1_0	Available
☐	IntelCoreWebAPI2	for demo	api-intelcore2	REST	Regional	2026-05-16	TLS_1_0	Available
☐	mydemo1		api-demo2	REST	Regional	2025-04-28	TLS_1_0	Available

- Choose **HTTP API**.
- Click **Build**.

The screenshot shows the 'Create API' wizard in the AWS API Gateway console. The 'Choose an API type' step is active. The 'HTTP API' option is selected and highlighted with a red box. The 'Build' button for the HTTP API is also highlighted with a red box.

Choose an API type [Info](#)

HTTP API

Build low-latency and cost-effective REST APIs with built-in features such as OIDC and OAuth2, and native CORS support.

Works with the following:
Lambda, HTTP backends

[Import](#) **Build**

WebSocket API

Build a WebSocket API using persistent connections for real-time use cases such as chat applications or dashboards.

Works with the following:
Lambda, HTTP, AWS Services

Build

REST API

Develop a REST API where you gain complete control over the request and response along with API management capabilities.

Step 2 – Create the HTTP API

- Enter the API name: UserAPI
- Keep the default settings.

Step 1: **Configure API**

Step 2 - optional: Configure routes

Step 3 - optional: Define stages

Step 4: Review and create

Configure API

API details

API name
An HTTP API must have a name. The name is a non-unique value you use to identify and organize your APIs. To programmatically refer to this API, use the API ID that API Gateway generates for you.

UserAPI

IP address type [Info](#)
Select the type of IP addresses that can invoke the default endpoint for your API. You don't need to redeploy your API for the update to take effect.

☒ **IPv4**
Includes only IPv4 addresses.

☐ **Dualstack**
Includes IPv4 and IPv6 addresses.

Integrations (0) [Info](#)
Specify the backend services that your API will communicate with. These are called integrations. For a Lambda integration, API Gateway invokes the Lambda function and responds with the response from the function. For an HTTP integration, API Gateway sends the request to the URL that you specify and returns the response from the URL.

[Add integration](#)

[Cancel](#) [Review and create](#) **[Next](#)**

CloudShell Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

- Leave routes and stages unchanged for now.
- Click Next.

Step 1: Configure API

Step 2 - optional: Configure routes

Step 3 - optional: **Define stages**

Step 4: Review and create

Define stages - optional

Configure stages [Info](#)

Stages are independently configurable environments that your API can be deployed to. You must deploy to a stage for API configuration changes to take effect, unless that stage is configured to autodeploy. By default, all HTTP APIs created through the console have a default stage named \$default. All changes that you make to your API are autodeployed to that stage. You can add stages that represent environments such as development or production.

Stage name

\$default

Auto-deploy

☒

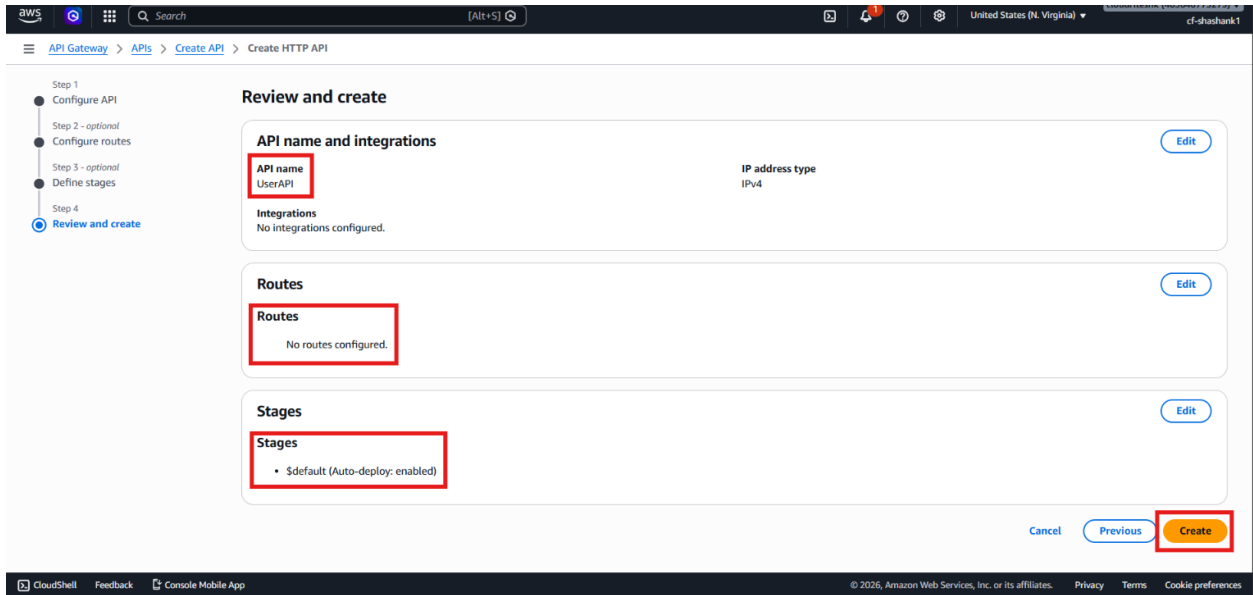
[Remove](#)

[Add stage](#)

[Cancel](#) [Previous](#) **[Next](#)**

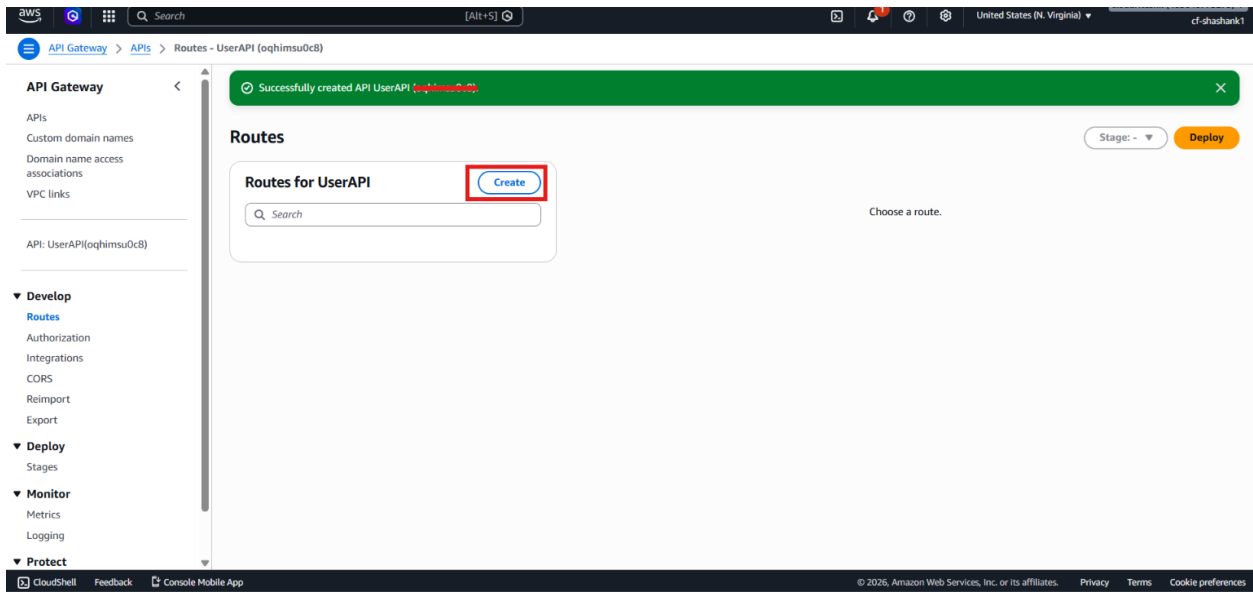
CloudShell Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

- Click Create.

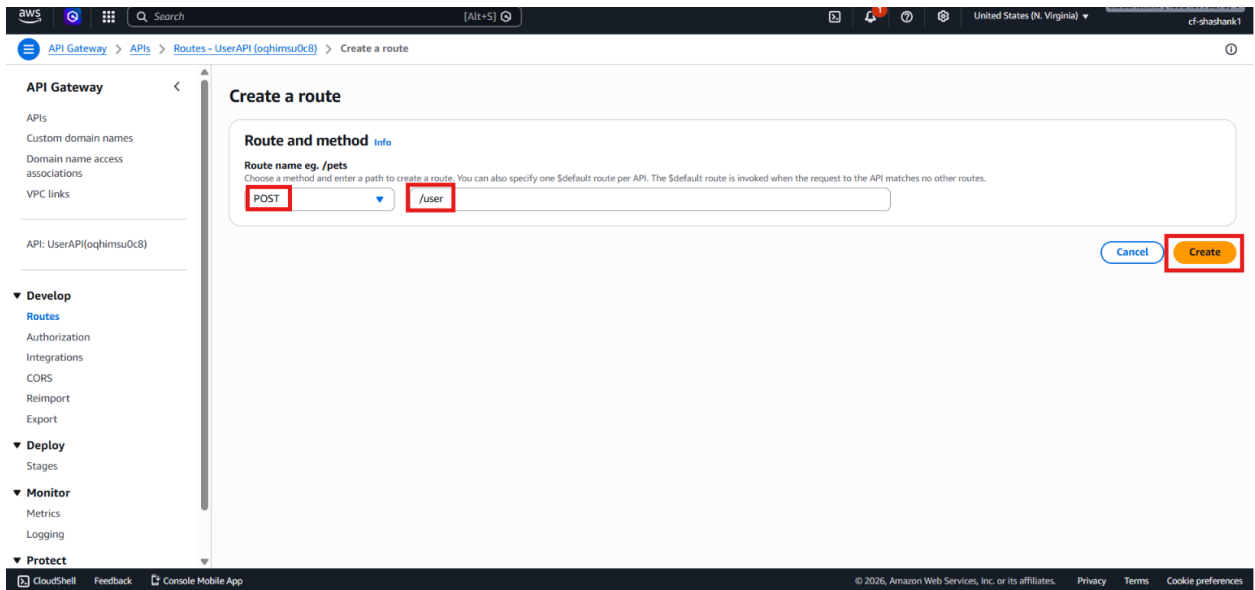


Step 3 – Create the Create User Route

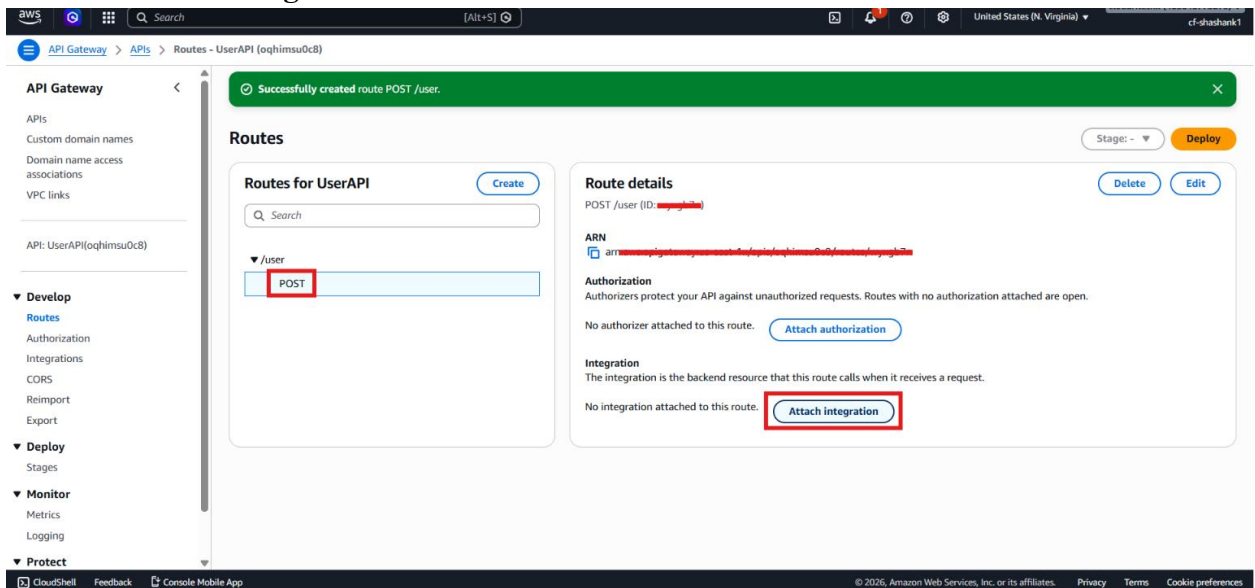
- Open UserAPI.
- Go to **Routes**.
- Click **Create**.



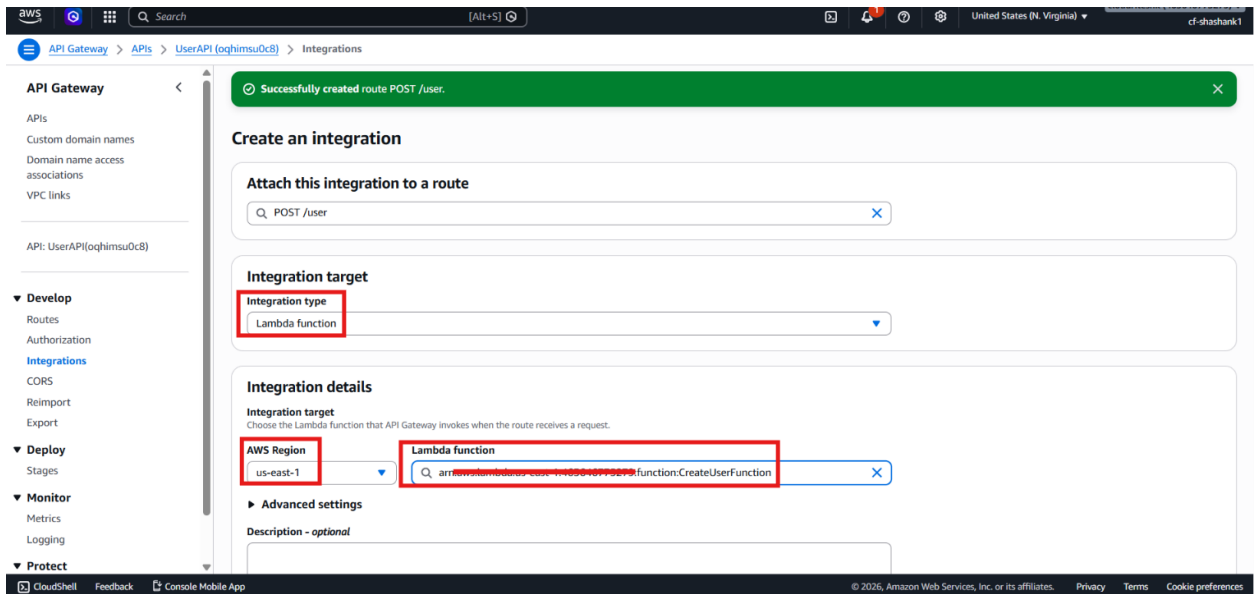
- Select **POST** as the method.
- Enter the route path: `/user`
- Click **Create**.



- Click **Attach integration**.



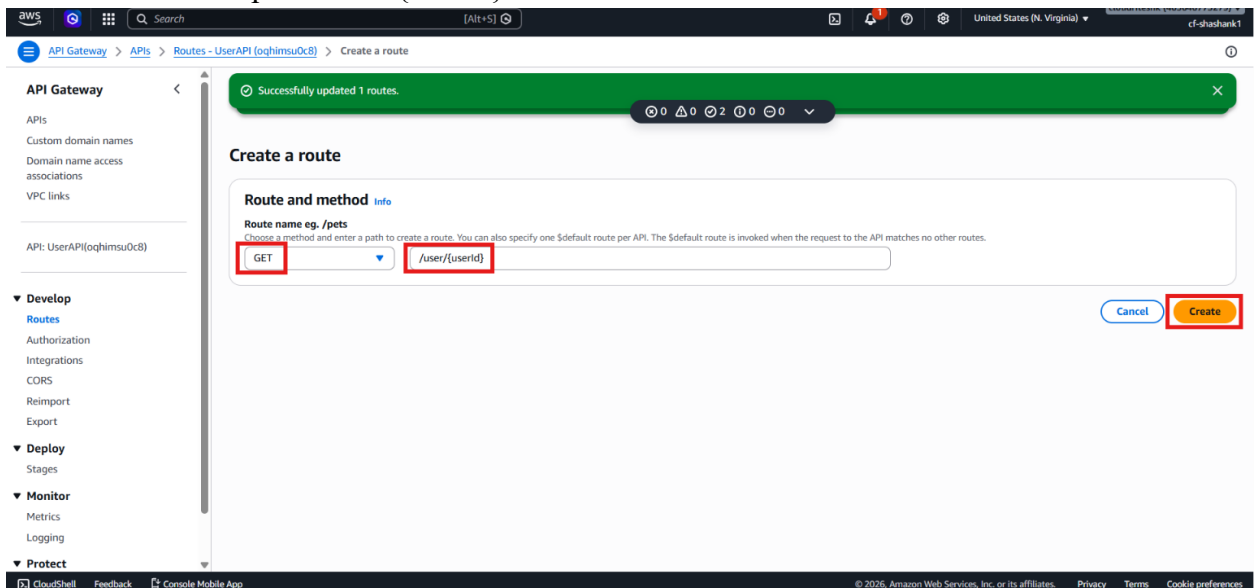
- Choose **Lambda function**.
- Select **createUserFunction**.



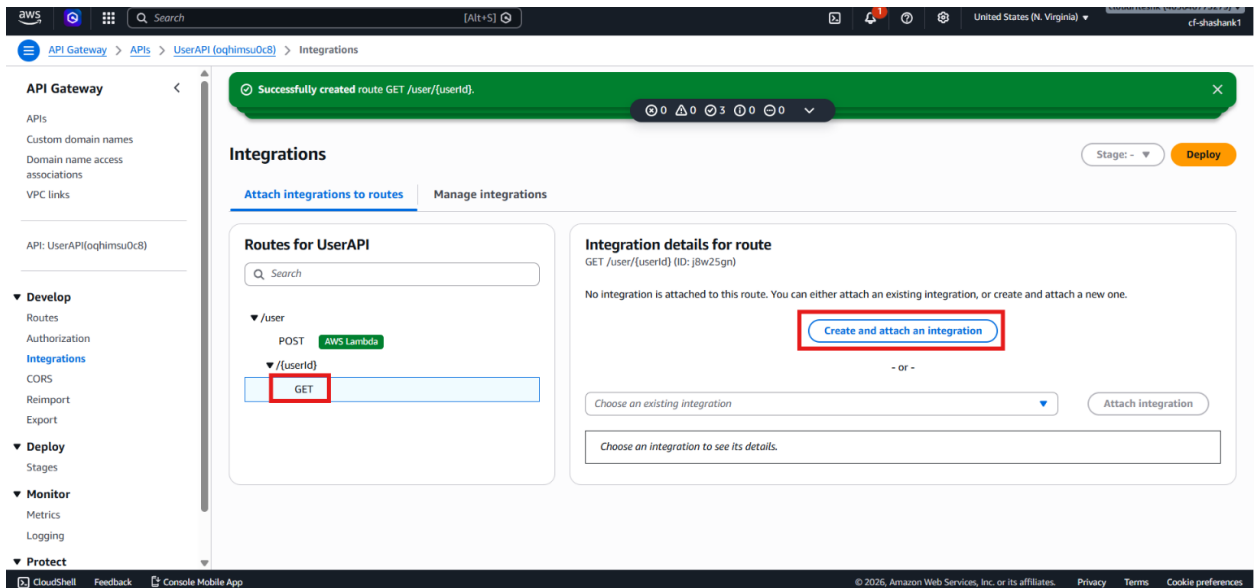
- Click **Create**.

Step 4 – Create the Get User Route

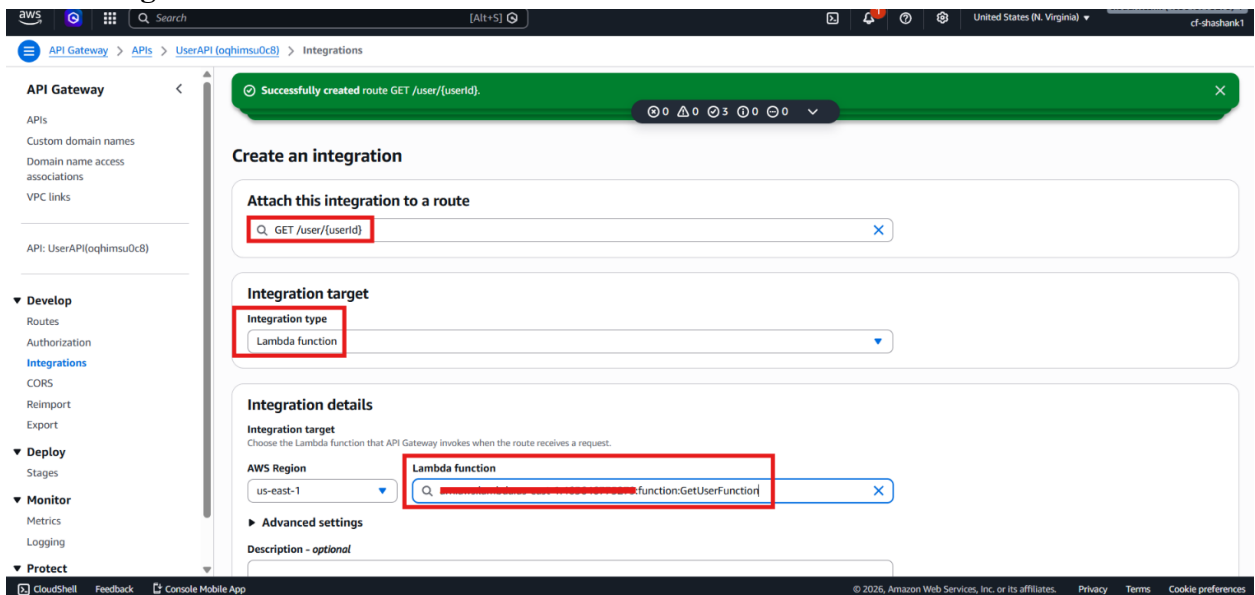
- Go back to **Routes**.
- Click **Create**.
- Select **GET** as the method.
- Enter the route path: `/user/{userId}`



- Click **Create**.
- Click **Attach integration**.



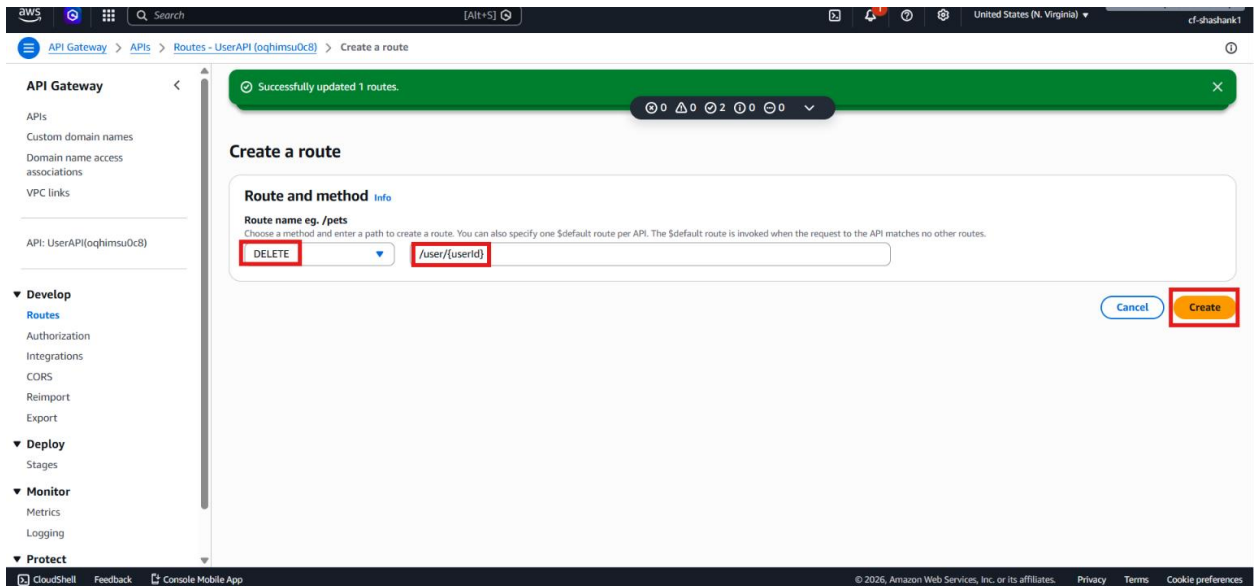
- Choose **Lambda function**.
- Select **getUserFunction**.



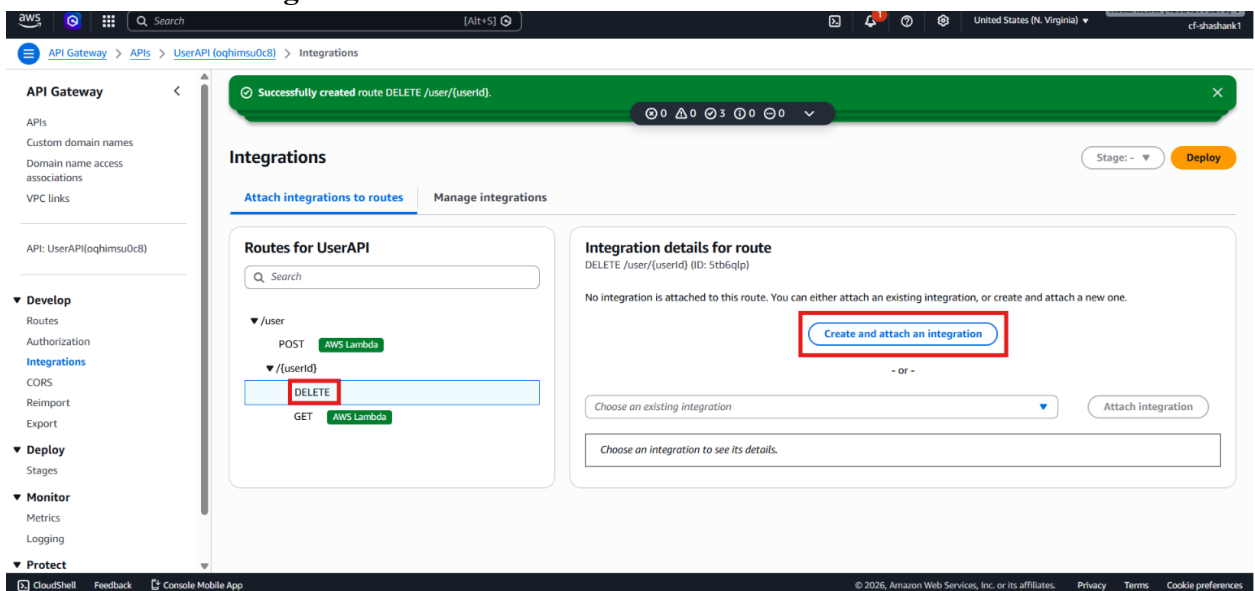
- Click **Create**.

Step 5 – Create the Delete User Route

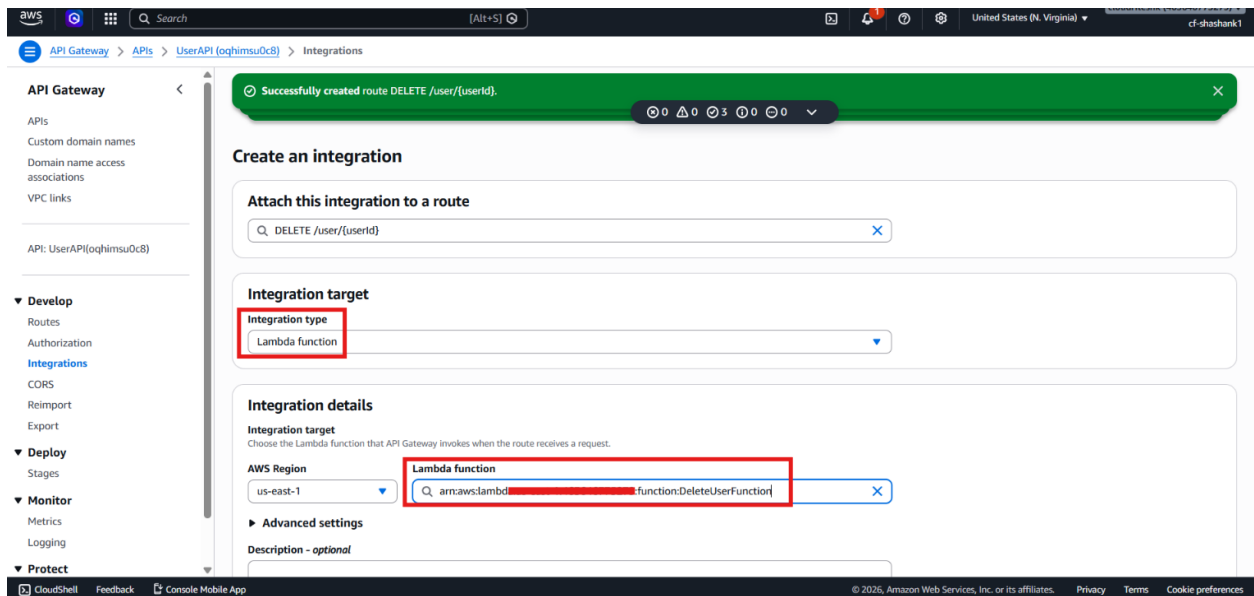
- Go back to **Routes**.
- Click **Create**.
- Select **DELETE** as the method.
- Enter the route path: `/user/{userId}`



- Click **Create**.
- Click **Attach integration**.



- Choose **Lambda function**.
- Select **deleteUserFunction**.



- Click **Create**.

Step 6 – Review the API Mapping

- Confirm the route mappings:
 - POST /user → **createUserFunction**
 - GET /user/{userId} → **getUserFunction**
 - DELETE /user/{userId} → **deleteUserFunction**
- Review the API trigger URL.
- Note that this is the URL the frontend or Postman will use to invoke the API.

Step 7 – Prepare for Testing

- Open the **Stages** tab.
- Copy the invoke URL.
- Keep the API ready for the final testing step.
- Prepare to use a frontend app or Postman in the next lab part.

Verification

- The **HTTP API** was created successfully in **API Gateway**.
- The route structure for create, read, and delete was defined.
- Each route was connected to the correct **AWS Lambda** function.
- The API is now acting as the bridge between the frontend and Lambda backend.
- The lab is ready for end-to-end testing in the next part